

Decision Log - Delta Support Agent

The non-obvious decisions I made and why.

1. **I chose Delta as the brand** because its Twitter threads were relatively clean to reconstruct, and only around 16% of messages were deflected to DMs. This meant I could get actual customer-brand resolution pairs for grounding instead of a dataset full of “please DM us” dead ends. A brand with heavy DM deflection would have made the retrieval corpus much less useful.
2. **I defined the intents from the actual data**, rather than starting with a generic customer-support taxonomy. I read and clustered Delta's messages and came up with 11 intents based on what customers were actually asking about, such as disruptions, baggage, SkyMiles, bookings, etc.
3. **I reduced 14 raw labels to 11 canonical intents**. Some labels were essentially describing the same thing, such as `booking_change`, `booking_issue`, and `fare_fares`, which I merged into `booking_fare`. With a relatively small dataset, keeping these separate would have increased sparsity and confusion without adding much useful information.
4. **I used TF-IDF + Logistic Regression for the main intent classifier instead of an LLM**. I initially tested few-shot LLM classification, but on the golden set it did not outperform the TF-IDF model (macro-F1 around 0.43). TF-IDF is also free, deterministic, fast, and reproducible, which makes it much easier for a grader to run and verify.
5. **I used weak supervision to bootstrap the classifier**. Instead of training only on the 220 manually labelled examples, I created keyword-based rules that automatically labelled around 7k additional messages. Training only on 220 examples resulted in low-confidence predictions (~0.20), while adding the weakly labelled data increased live confidence to around 0.75 and improved macro-F1.
6. **I gave the 220 gold labels 10x more weight than the weak labels**. The idea was to trust the human-labelled data more than the noisy keyword labels, while still getting the benefit of having thousands of training examples. It's essentially a practical way to combine a small amount of clean data with a large amount of noisy data.
7. **The escalation policy follows “escalate unless proven safe.”** `AUTO_OK` is intentionally a small allow-list covering five mainly informational intents. Anything account-specific or transaction-specific is escalated. The goal is to avoid confidently answering something the agent shouldn't handle, even if that means some safe queries get escalated unnecessarily.
8. **Replies are grounded in Delta's historical responses rather than being generated completely from scratch**. The system retrieves relevant historical Delta replies and uses them as the source for the response. This reduces the chance of the LLM inventing policies, fees, or promises that Delta never actually made.
9. **I set the retrieval grounding threshold at 0.38 based on calibration rather than guessing**. I looked at cosine similarity scores and found that genuinely answerable queries generally retrieved in the ~0.40–0.62 range, while unrelated queries were closer to ~0.30. A 0.38 threshold gave a useful separation. Below that, the system escalates instead of using a potentially unrelated historical reply. This also fixed an actual failure case where a credit-card question was incorrectly answered using a cancellation-related response.
10. **I restricted the LLM to phrasing the final reply**. It does not make the routing or classification decision. This is intentional because an incorrect routing decision is more serious than slightly awkward wording. I also made the LLM path fail-safe: if the API call fails, for example because credits are exhausted, the system falls back to retrieval/templates instead of producing a blank response or making a wrong classification.
11. **I added a high-precision brand-term override on top of the classifier**. Terms such as `SkyMiles`, `Medallion`, and `SkyClub` strongly indicate the loyalty intent, so they override the classifier when detected. This helps with cases where the linear bag-of-

words model gets distracted by another word in the message — for example, “SkyMiles for a checked bag” could otherwise be classified as baggage.

12. **I implemented a lightweight, negation-aware sentiment check instead of adding another dependency.** It handles cases such as “not impressed” or “wasn't good” by normalizing `n't` $\rightarrow$ `not` and checking a small 3-token window. It's deterministic, easy to explain, and avoids adding a separate sentiment library just for this task.
13. **I validated the LLM-as-judge before relying on its scores.** I compared the judge's ratings against human-labelled examples and measured agreement using **Cohen's κ** . This gives evidence that the judge is reasonably aligned with human evaluation rather than blindly trusting an LLM-generated metric.